

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Аппаратно-программное обеспечение ЭВМ и сетей»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства
Intel – обработка символьных данных»

Выполнила:

Студентка 4 курса

Группы АС-66

Прокурат В.Д.

Проверила:

Кулик А.Д.

Цель работы: изучение программной модели микропроцессора, практическая работа с подсистемой регистровой и оперативной памяти.

Вариант 20 (4)

Постановка задачи. Дан текст – непустая последовательность не длиннее ста символов. Признаком конца ввода является точка, в сам текст точка не входит.

Проверить, удовлетворяет ли текст заданному условию. Если условие выполнено, преобразовать текст по указанному в варианте правилу; в противном случае выдать соответствующее сообщение. Преобразованный текст напечатать.

Проверяемое условие и правила обработки текста определяются конкретным вариантом задания.

Если введенная последовательность символов не является текстом, преобразовывать ее не нужно, а следует напечатать соответствующее сообщение.

Ввод текста, проверка условия, обработка текста и печать результата должны выполняться последовательно, отдельными частями программы.

Используя знания и навыки, полученные в результате выполнения лабораторной работы №1, а также изучив дополнительные теоретические материалы, приведенные в методическом указании, написать программу на ассемблере, реализующую обработку текста согласно варианту, выданному преподавателем.

Проверяемое условие: текст содержит не менее трех латинских букв.

Правило преобразования: заменить каждую прописную латинскую букву следующей за ней по алфавиту, букву Z менять на A.

Требования к программе: нельзя использовать дополнительную память, размер которой зависит от длины текста (в частности, стек) во всех вариантах, кроме тех, где явно не указано обратное.

При реализации вариантов, помеченных "с.к.", требуется использовать строковые команды с префиксами повторения.

Проверку условия и выполнение преобразований текста следует реализовать в виде процедур. Глобальные переменные не использовать.

Интерфейс программы должен быть достаточно удобным: надо печатать приглашение для ввода, пояснять, какое условие проверяется, истинно ли оно и как текст преобразуется.

Рекомендации: в некоторых вариантах удобно реализовать в качестве процедур или макросов проверку, является ли символ буквой, цифрой и т.п.

В процедуре преобразования текста может быть полезным иметь два указателя на позиции в тексте: откуда происходит чтение и куда идёт запись.

Максимально возможную длину текста описать в виде константы.

Код программы:

1. Ввод текста до точки. Ищет символ «.» во введённой строке и заменяет его на нуль-терминатор — так получается С-строка для дальнейшей обработки.

```
asm
    mov ecx, nRead
    lea esi, buffer
    xor edx, edx
findLoop:
    cmp edx, ecx
    jge noDot
    cmp byte ptr [esi + edx], '.'
    je foundDot
    inc edx
    jmp findLoop

foundDot:
    mov byte ptr [esi + edx], 0
    mov buflen, edx
    jmp afterFind
```

2. Проверка условия «не менее трёх латинских букв». Каждый символ проверяется на принадлежность диапазонам A..Z или a..z. Проверка сделана через вычитание: считаем смещение от 'A' (или 'a') и смотрим, укладывается ли оно в диапазон 0..25. Если символ — буква, счётчик ecx увеличивается. В конце счётчик сравнивается с 3.

```
asm
ccLoop:
    mov al, [esi + edx]
    test al, al
    jz ccDone

    mov bl, al
    sub bl, 'A'
    cmp bl, 25
    jbe ccLetter

    mov bl, al
    sub bl, 'a'
    cmp bl, 25
    ja ccNext

ccLetter:
    inc ecx
ccNext:
    inc edx
    jmp ccLoop

ccDone:
    xor eax, eax
    cmp ecx, 3
    jl ccRet
    mov eax, 1
```

3. Преобразование прописных латинских букв. Для каждой буквы в диапазоне A..Z её код увеличивается на 1. Если символ равен Z, он заменяется на A — это и есть циклический переход. Строчные латинские буквы и остальные символы остаются без изменений.

```
asm
ttLoop:
    mov al, [esi + ecx]
    test al, al
    jz ttDone

    cmp al, 'A'
    jb ttNext
    cmp al, 'Z'
    ja ttNext

    cmp al, 'z'
    je ttWrap
    inc al
    mov [edi + ecx], al
    jmp ttNext
```

```

ttWrap:
    mov al, 'A'
    mov [edi + ecx], al

ttNext:
    inc ecx
    jmp ttLoop

```

4. Вывод сообщения о результате проверки. В зависимости от возвращённого значения checkCondition (1 или 0) программа печатает соответствующее сообщение и либо выполняет преобразование, либо завершается.

```

asm
    push offset buffer
    call checkCondition
    add esp, 4
    cmp eax, 0
    je failBranch

    mov edx, offset sOK
    mov ecx, sOKLen
    call WriteStr

    push offset buffer
    call transformText
    add esp, 4
    ; ...
failBranch:
    mov edx, offset sFail
    mov ecx, sFailLen
    call WriteStr
    push 0
    call ExitProcess

```

5. Печать строки через WinAPI. Вспомогательная процедура, которой пользуются все остальные части программы. Демонстрирует работу с консолью через WriteConsoleA — без CRT, только WinAPI.

```

asm
WriteStr proc uses eax
    push 0
    push offset nWrite
    push ecx
    push edx
    push hOut
    call WriteConsoleA
    ret
WriteStr endp

```

Результат работы программы:

```

D:\politex\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab2>ml /coff /Zi lab2.asm /link /subsystem:console /entry:start
Microsoft (R) Macro Assembler Version 14.51.36257.0
Copyright (C) Microsoft Corporation. All rights reserved.

Assembling: lab2.asm
Microsoft (R) Incremental Linker Version 14.51.36257.0
Copyright (C) Microsoft Corporation. All rights reserved.

/DEBUG
/OUT:lab2.exe
lab2.obj
/subsystem:console
/entry:start

D:\politex\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab2>lab2.exe
Please type a line (max 100 chars, finish with a dot):
Hello World.

Checking rule: at least 3 letters from the Latin alphabet.
Rule passed. Uppercase Latin letters are shifted one forward, Z becomes A.
Result: Iello Xorld

```

```
D:\politex\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab2>lab2.exe
Please type a line (max 100 chars, finish with a dot):
OpQrNoJkTuQrNoQr ZZZ.

Checking rule: at least 3 letters from the Latin alphabet.
Rule passed. Uppercase Latin letters are shifted one forward, Z becomes A.
Result: PpRrOoKkUuRrOoRr AAA
```

```
D:\politex\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab2>lab2.exe
Please type a line (max 100 chars, finish with a dot):
1234567890.

Checking rule: at least 3 letters from the Latin alphabet.
Rule failed: fewer than 3 Latin letters were found. No changes made.
```

Вывод: в ходе работы изучены регистры процессора Intel x86, принципы организации памяти и обработку символьных данных. Была написана программа на ассемблере MASM с использованием функций WinAPI: реализованы ввод текста до точки, проверка условия (не менее трёх латинских букв) и замена прописных латинских букв на следующую по алфавиту с переходом Z → A. Проверка условия и преобразование оформлены отдельными процедурами. Правильность работы программы подтверждается скриншотами запусков с разными исходными текстами. В итоге я убедилась, что ассемблер даёт прямой доступ к регистрам и памяти, но при этом требует внимательности: границы буфера и соглашения вызова приходится контролировать вручную.